

API Versioning & Lifecycle Management

Nhóm 1

23020094 Tôn Thiện Khỏe
21020123 Nguyễn Tiến Hoàng
2302014 Hà Vũ Công
22024538 Trần Hữu Mạnh

Bài Thuyết Trình

7 tháng 5, 2026

Bài toán thực tế (Scenario)

Hiện trạng hệ thống

Một hệ thống thanh toán đang sử dụng API:

POST /api/v1/payments

Đang được sử dụng bởi:

- Web Frontend
- Mobile Application
- Banking Partners
- Third-party Services

Yêu cầu mới (Business)

Cần hỗ trợ thêm: E-wallet, QR Code, Multi-currency, Token-based security, Transaction metadata.

Vấn đề phát sinh (Nếu sửa trực tiếp)

- ✗ Frontend cũ có thể lỗi
- ✗ Mobile app cũ có thể crash
- ✗ Đối tác tích hợp bị fail
- ✗ Production có nguy cơ downtime

Kết luận & Giải pháp

Hệ thống cần:

- ✓ **API Versioning**
- ✓ **Backward Compatibility**
- ✓ **Deprecation Strategy**
- ✓ **Migration Plan**

→ *Nâng cấp an toàn, không ảnh hưởng client*

API Lifecycle Management

Design → **Development** → **Release** → **Maintenance** → **Deprecation** → **Sunset**

- **Design:** Thiết kế cấu trúc và tiêu chuẩn API.
- **Development:** Triển khai và kiểm thử chức năng API.
- **Release:** Phát hành API cho client sử dụng.
- **Maintenance:** Bảo trì, sửa lỗi và tối ưu hệ thống.
- **Deprecation:** Thông báo API sẽ ngừng hỗ trợ trong tương lai.
- **Sunset (Retirement):** Ngừng hoạt động hoàn toàn phiên bản API cũ.

Mục tiêu

API không phải "viết một lần dùng mãi mãi". Việc quản lý vòng đời giúp hệ thống vận hành trơn tru qua các đợt nâng cấp.

Breaking Change vs Non-breaking Change

Non-breaking Change

Mô tả: Thêm dữ liệu mới nhưng không làm ảnh hưởng client cũ.

Ví dụ:

```
{  
  "name": "John",  
  "phone": "123456" // Thêm mới  
}
```

Đặc điểm:

- ✓ Client cũ vẫn hoạt động
- ✓ Tương thích ngược (Backward compatible)
- ✓ An toàn khi nâng cấp

Breaking Change

Mô tả: Thay đổi làm client cũ không còn hoạt động đúng.

Ví dụ (Từ v1 lên v2):

```
v1: { "fullName": "John Doe" }  
    |  
    v  
v2: { "firstName": "John",  
      "lastName": "Doe" }
```

Hậu quả:

- ✗ Frontend/Mobile app có thể crash
- ✗ Third-party integration thất bại

Làm thế nào để hệ thống biết Client đang gọi phiên bản API nào?

1. URL Versioning

Đặt version trực tiếp trong đường dẫn

`/api/v1/users`

2. Header Versioning

Truyền version qua HTTP Header

`Accept: version=v1`

3. Query Parameter

Truyền version qua Query Parameter

`/users?version=1`

1. URL Versioning

Khái niệm & Hoạt động: Đặt phiên bản trực tiếp trong URL. Server định tuyến request dựa trên URL endpoint.

Ví dụ

Version 1

```
GET /api/v1/users
{ "fullName": "John Doe" }
```

Version 2

```
GET /api/v2/users
{ "firstName": "John",
  "lastName": "Doe" }
```

Thực tế: Phổ biến nhất (REST API, Spring Boot, NodeJS).

Ưu điểm

- ✓ Dễ hiểu, dễ đọc, dễ debug qua trình duyệt.
- ✓ Dễ định tuyến (routing) qua API Gateway.
- ✓ Dễ maintain nhiều version.

Nhược điểm

- ✗ URL dài hơn, duplicate endpoints.
- ✗ Khó quản lý khi có quá nhiều version.

2. Header Versioning

Khái niệm & Hoạt động: Client gửi thông tin version trong HTTP Header. Server đọc header để xử lý tương ứng.

Ví dụ

Version 1

```
GET /api/users
```

```
Accept: app/vnd.v1+json
```

```
{ "fullName": "John Doe" }
```

Version 2

```
GET /api/users
```

```
Accept: app/vnd.v2+json
```

```
{ "firstName": "John"... }
```

Thực tế: Dùng trong các Public APIs lớn, Microservices yêu cầu URL sạch.

Ưu điểm

- ✓ URL gọn gàng, không đổi cấu trúc.
- ✓ Chuẩn RESTful hơn.
- ✓ Phù hợp large-scale APIs.

Nhược điểm

- ✗ Khó test trực tiếp trên trình duyệt.
- ✗ Frontend phải cấu hình header.
- ✗ Dev mới khó tiếp cận hơn.

3. Query Parameter Versioning

Khái niệm & Hoạt động: Truyền phiên bản thông qua query parameter ('?version=x'). Server rẽ nhánh logic theo giá trị này.

Ví dụ

Version 1

```
GET /api/users?version=1
{ "fullName": "John Doe" }
```

Version 2

```
GET /api/users?version=2
{ "firstName": "John",
  "lastName": "Doe" }
```

Thực tế: Phù hợp hệ thống nhỏ, internal tools, prototype (Ít dùng cho Enterprise).

Ưu điểm

- ✓ Rất dễ implement và test nhanh.
- ✓ Không thay đổi cấu trúc đường dẫn chính.

Nhược điểm

- ✗ Không clean, không chuẩn RESTful.
- ✗ Dễ gây nhầm lẫn nếu có nhiều parameters khác.
- ✗ Khó maintain lâu dài.

So sánh các chiến lược Versioning

Tiêu chí	URL Versioning	Header Versioning	Query Parameter
Cách hoạt động	Version trong URL	Version trong Header	Version trong Query
Ví dụ	/api/v1/users	Accept: v1	?version=1
Dễ hiểu / Dễ debug	Cao	Thấp	Cao
Chuẩn RESTful	Trung bình	Cao	Thấp
Dễ maintain	Cao	Trung bình	Thấp
Độ phổ biến	Rất cao	Trung bình	Thấp
Enterprise usage	Phổ biến nhất	Có sử dụng	Ít dùng

Kết luận

Trong thực tế, **URL Versioning** là chiến lược được sử dụng phổ biến nhất nhờ tính đơn giản, dễ maintain và dễ debug.

Phân tích sâu về Breaking Changes

Định nghĩa: Thay đổi cấu trúc request/response khiến client cũ không thể xử lý. Nếu xảy ra trên production, toàn bộ client cũ có thể lỗi đồng loạt.

Ví dụ 1: Rename Field

v1: { "fullName": "John" }
v2: { "firstName": "John" }
→ *Frontend gọi fullName bị crash.*

Ví dụ 2: Change Data Type

v1: { "price": "100" } (*string*)
v2: { "price": 100 } (*number*)
→ *App deserialize dữ liệu bị fail.*

Loại thay đổi	Breaking?
Rename field	Yes
Remove field	Yes
Change datatype	Yes
Remove endpoint	Yes
Change authentication	Yes
Add optional field	No

Ý tưởng cốt lõi

Không bao giờ triển khai trực tiếp Breaking Changes lên Production API đang có Client sử dụng. Enterprise ưu tiên **Sự ổn định (Stability)** hơn tốc độ thay đổi.

- **1. Tạo API Version mới:** Thay vì sửa v1, tạo hẳn v2. Client cũ vẫn chạy v1, client mới dùng v2.
- **2. Maintain Backward Compatibility:** Giữ tính tương thích ngược. Ưu tiên thêm optional fields thay vì xóa/đổi tên trường cũ.
- **3. Hỗ trợ đa phiên bản (Multiple Versions):** Chạy song song v1 và v2 trên server trong một thời gian để client có thời gian nâng cấp.
- **4. Cung cấp Migration Guide:** Cung cấp tài liệu rõ ràng chỉ ra sự thay đổi (VD: *fullName* → *firstName*) để dev nâng cấp dễ dàng.

API Deprecation (Ngừng hỗ trợ an toàn)

Deprecation là đánh dấu API cũ sẽ ngừng hỗ trợ trong tương lai, nhưng **hiện tại vẫn hoạt động** để client có thời gian di chuyển (migrate) an toàn.

Deprecation Lifecycle

1. Release API v2
2. **Deprecate API v1** (Ra thông báo)
3. Gửi cảnh báo đến Developers
4. Khoảng thời gian Migration
5. **Sunset v1** (Chính thức tắt v1)

Cách thông báo Deprecation

- **Tài liệu API:** Gắn tag *Deprecated*.
- **HTTP Headers (Response):**
Deprecation: true
Sunset: 2026-12-30
- **Email / Dev Portal:** Gửi timeline và Migration guide.

→ *Giúp hệ thống nâng cấp an toàn, Zero Downtime, không làm gián đoạn Business.*

Cảm ơn thầy và các bạn đã lắng nghe!

Q & A